

MiauChat

1. Scopul proiectului

Proiectul Miauchat este o aplicație client-server care permite utilizatorilor să comunice în timp real printr-o interfață grafică simplă și intuitivă. Aplicația implementează funcții esențiale pentru mesagerie instantanee, precum conectarea mai multor utilizatori la un server comun, trimiterea și primirea de mesaje în timp real, gestionarea unui istoric al conversațiilor și notificări audio pentru mesajele noi.

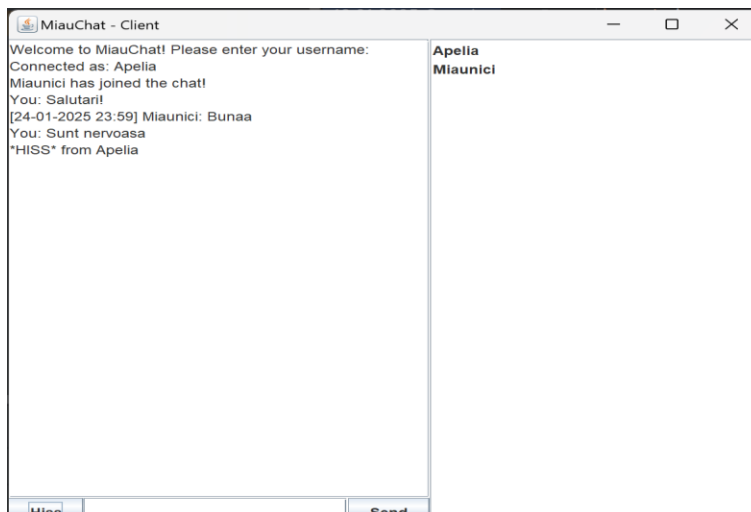
Aplicația folosește arhitectura client-server, serializarea datelor, socketuri, fire de execuție și sincronizare pentru a asigura o execuție corectă și performantă.

2. Structura proiectului

Proiectul este organizat pe trei module principale. Modulul Client conține clasa ChatClient, care implementează logica clientului, inclusiv interfața grafică pentru utilizator și gestionarea conexiunii la server. Modulul Server include clasele ChatServer, ClientHandler și Message. Clasa ChatServer asigură gestionarea conexiunilor și coordonarea traficului de mesaje, ClientHandler gestionează conexiunea individuală cu fiecare client, iar Message reprezintă obiectul mesaj care este trimis între client și server. Modulul Utils include clasa SoundUtils, care oferă funcții pentru redarea notificărilor audio (ex. sunetul "meow", hiss).

3. Funcționalități implementate

Clientul se conectează la server utilizând socketuri. După conectare, utilizatorului i se afișează o interfață grafică care include o zonă de chat pentru mesaje, o listă a utilizatorilor conectați, un câmp de text pentru introducerea mesajelor și un buton pentru trimiterea mesajelor. De asemenea, clientul afișează un fundal grafic personalizat, utilizând clasa BackgroundPanel. Mesajele trimise sunt afișate local cu prefixul "You: ...", în timp ce mesajele primite de la ceilalți utilizatori sunt afișate în zona de chat și sunt acompaniate de un sunet de notificare. La fiecare mesaj primit, aplicația redă un fișier audio. Comanda specială Hiss, accesibilă printr-un buton dedicat, declanșează un sunet specific și afișează numele expeditorului în chat (ex. *HISS* from Alice).



Serverul ascultă conexiuni pe un port prestabilit (12345) și acceptă conexiuni multiple de la clienți folosind socketuri. Pentru fiecare client conectat, serverul creează un fir de execuție separat, permițând funcționarea simultană a mai multor utilizatori. Resursele partajate, precum lista de mesaje (chatHistory) și lista clienților conectați (clientHandlers), sunt sincronizate folosind blocuri synchronized pentru a preveni accesul simultan conflictual. În plus, comanda Hiss utilizează un mutex (hissLock) pentru a asigura că doar un client poate trimite comanda la un moment dat. Mesajele primite de la un client sunt adăugate în istoricul de mesaje și transmise tuturor celorlalți clienți. De asemenea, istoricul conversațiilor este salvat într-un fișier serializat (chat_history.ser) și încărcat automat la pornirea serverului. Lista utilizatorilor conectați este actualizată periodic și transmisă tuturor clienților.

4. Arhitectura proiectului

Fluxul general de execuție al aplicației este următorul. Serverul pornește și ascultă conexiuni pe portul prestabilit, încărcând istoricul conversațiilor, dacă acesta există. Un client se conectează la server, introduce numele de utilizator și primește confirmarea conectării și istoricul conversațiilor. Mesajele trimise de un client sunt transmise tuturor celorlalți utilizatori conectați, fiind salvate în istoricul conversațiilor pe server. Pentru a proteja accesul la resursele partajate, aplicația implementează **programare concurentă**: listele partajate sunt sincronizate, iar comanda Hiss este gestionată printr-un **mutex** pentru a evita interferențele. La deconectarea unui client, serverul actualizează lista utilizatorilor conectați, notificând toți ceilalți clienți.

5. Exemple de cod relevante

- Conectarea unui client la server: Atunci când clientul este lansat, acesta creează o conexiune către server folosind un socket.

```
public ChatClient() { 1 usage  ⤴ Ion Apelia Cosmina
    initializeGUI();

    try {
        socket = new Socket(SERVER_ADDRESS, SERVER_PORT);
        out = new PrintWriter(socket.getOutputStream(), autoFlush: true);
        in = new BufferedReader(new InputStreamReader(socket.getInputStream()));

        new Thread(this::listenForMessages).start();
    } catch (IOException e) {
        showError( errorMessage: "Unable to connect to the server: " + e.getMessage());
    }
}
```

- Trimiterea unui mesaj text de la client la server: Atunci când utilizatorul introduce un mesaj și apasă pe butonul de trimitere, mesajul este transmis serverului.

```
82     private void sendMessage() { 2 usages  ⚙ Ion Apelia Cosmina *
83         String message = messageField.getText().trim();
84         if (!message.isEmpty()) {
85             out.println(message);
86             if (clientName != null) {
87                 chatArea.append("You: " + message + "\n");
88             }
89             messageField.setText("");
90         }
91     }
```

- Gestionarea mesajelor pe server: Serverul primește mesajele trimise de client și le retransmite către toți ceilalți clienți.

```
79     public static void broadcastMessage(Message message, ClientHandler sender) { 1 usage  ⚙ Ion Apelia Cosmina
80         synchronized (chatHistory) {
81             chatHistory.add(message);
82         }
83         saveChatHistory();
84
85         synchronized (clientHandlers) {
86             for (ClientHandler clientHandler : clientHandlers) {
87                 if (clientHandler != sender) {
88                     clientHandler.sendMessage(message.toString());
89                 }
90             }
91         }
92     }
```

- **Sincronizare (synchronized):** Previne accesul concurent la lista de mesaje și la lista de clienți conectați.
 - Mesajele sunt salvate în istoricul conversațiilor (`chatHistory`) și transmise tuturor clienților, cu excepția expeditorului.
- Trimiterea comenzii Hiss: Comanda Hiss nu este tratată special: clientul o trimite către server, care o retransmite tuturor celorlalți clienți. Insa folosește un mutex pentru a asigura că doar un fir poate procesa comanda la un moment dat.

```
69     public static void broadcastCommand(String command) { 1 usage  new *
70         synchronized (hissLock) { // Ensure one hiss at a time
71             synchronized (clientHandlers) {
72                 for (ClientHandler clientHandler : clientHandlers) {
73                     clientHandler.sendMessage(command);
74                 }
75             }
76         }
77     }
```

6. Concluzii

Proiectul Miauchat demonstrează o utilizare complexă a arhitecturii client-server și a funcționalităților de bază din Java, incluzând utilizarea socketurilor pentru comunicarea în rețea locală, sincronizarea firelor de execuție pentru accesul la resurse comune, persistența datelor pentru istoricul conversațiilor și implementarea unei interfețe grafice utilizator-friendly. Această aplicație poate fi extinsă în viitor cu funcționalități suplimentare, precum suport pentru mesaje multimedia (imagini, fișiere), criptare pentru securitatea comunicațiilor sau implementarea unei baze de date pentru gestionarea utilizatorilor și a conversațiilor.